VIT BHOPAL UNIVERSITY

CONTEXT-AWARE ADAPTIVE PATIENT MONITORING AND RISK ESCALATION SYSTEM :-

Name :- Vansh Ahuja

Registration No. :- 25BHI10003

Course :- B.Tech CSE (Health Informatics)

Department :- SCAI

Subject :- Fundamentals In AI And ML

Faculty's Name :- Dr. Monica Sankat

Project Report

Context-Aware Adaptive Patient Monitoring and Risk Escalation System :-

Vansh Ahuja

25BHI10003

VIT BHOPAL

INTRODUCTION :-

In today's fast-paced world, early detection of health risks plays a crucial role in preventing serious medical conditions. However, most individuals lack the knowledge or tools to interpret their symptoms and vital signs correctly.

This project presents a ***Context-Aware Adaptive Patient Monitoring System*** *that uses* ***Machine Learning and Artificial Intelligence techniques*** *to analyze patient health data and provide meaningful insights. The system is designed to simulate a basic healthcare decision-support system that can assist users in understanding their health condition.*

Problem Statement :-

Many people ignore early symptoms such as mild fever, abnormal heart rate, or low oxygen levels due to lack of awareness. This delay can result in severe complications.

The problem addressed in this project is:

1. ***To develop a system that can intelligently analyze patient data and provide early warnings about potential health risks.***

OR

There is no easily accessible system for common users to understand the severity of their health condition based on basic parameters such as blood pressure, heart rate, and oxygen level.

The problem is to design a system that:

- ***Accepts patient health inputs***
- ***Predicts risk levels***
- ***Detects emergency conditions***
- ***Provides meaningful guidance***

Objectives :-

- ***Build a predictive model using ML***
- ***Apply rule-based logic for safety-critical decisions***
- ***Provide a risk score for better interpretation***
- ***Create a modular and scalable system***
- ***Demonstrate AI + ML integration***

Functional Requirements :-

1. ***Accept user input (health parameters)***
2. ***Predict risk level (Low/Medium/High)***
3. ***Apply decision logic***
4. ***Generate risk score***
5. ***Display recommendation***

Non Functional Requirements :-

1. ***Simple and user-friendly interface***
2. ***Fast execution***
3. ***Clear output representation***
4. ***Modular code structure***

System Overview :-

The system works in three layers:

1. ***Input Layer → User enters health data***
2. ***ML Layer → Predicts risk***
3. ***AI Layer → Refines and validates prediction***

Libraries Used & Justification :-

1. ***pandas***

   Used for:

   - ***Reading dataset (CSV)***
   - ***Data manipulation***

- ***Why used:***

  Efficient handling of structured data

2. ***Scikit-learn***

   Used for:

   - ***Training ML model (Decision Tree)***

- ***Why used:***

  Simple, reliable ML library suitable for beginners

3. ***pickle***

   Used for:

   - ***Saving trained model (model.pkl)***

- ***Why used:***

  Avoids retraining model every time

Module-Wise Code Explanation :-

1. data_generator.py

Function:

- ***Generates synthetic dataset***

What it does:

- ***Creates random patient data***
- ***Includes features like age, BP, heart rate, etc.***
- ***Saves into CSV***

2. model.py

Function:

- ***Trains machine learning model***

What it does:

- ***Reads dataset***
- ***Converts categorical data***
- ***Trains Decision Tree***
- ***Saves model as model.pkl***

3. predict.py

Function:

- ***Takes user input***
- ***Converts input into model format***
- ***Predicts risk***

4. decision_logic.py

Function:

- ***Applies rule-based AI logic***

What it does:

- ***Checks critical conditions***
- ***Adjusts ML output***
- ***Generates risk score***

5. main.py

Function:

- ***Integrates entire system***

What it does:

- ***Takes input***
- ***Calls ML model***
- ***Calls AI logic***
- ***Displays output***

System Architecture :-

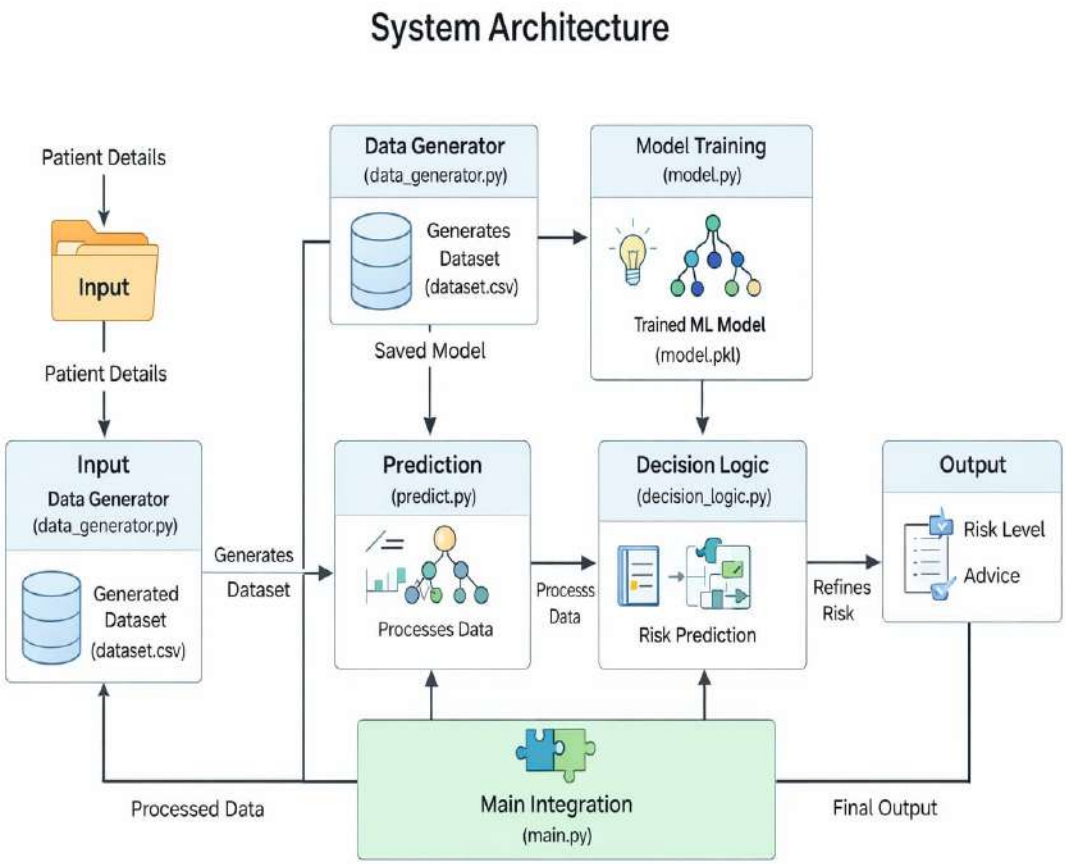
The system follows a modular architecture:

Input → ML Model → Decision Logic → Output

Modules:

- ***Data Generator***
- ***Model Training***
- ***Prediction***
- ***Decision Logic***
- ***Main Integration***

System Architecture Diagram :-

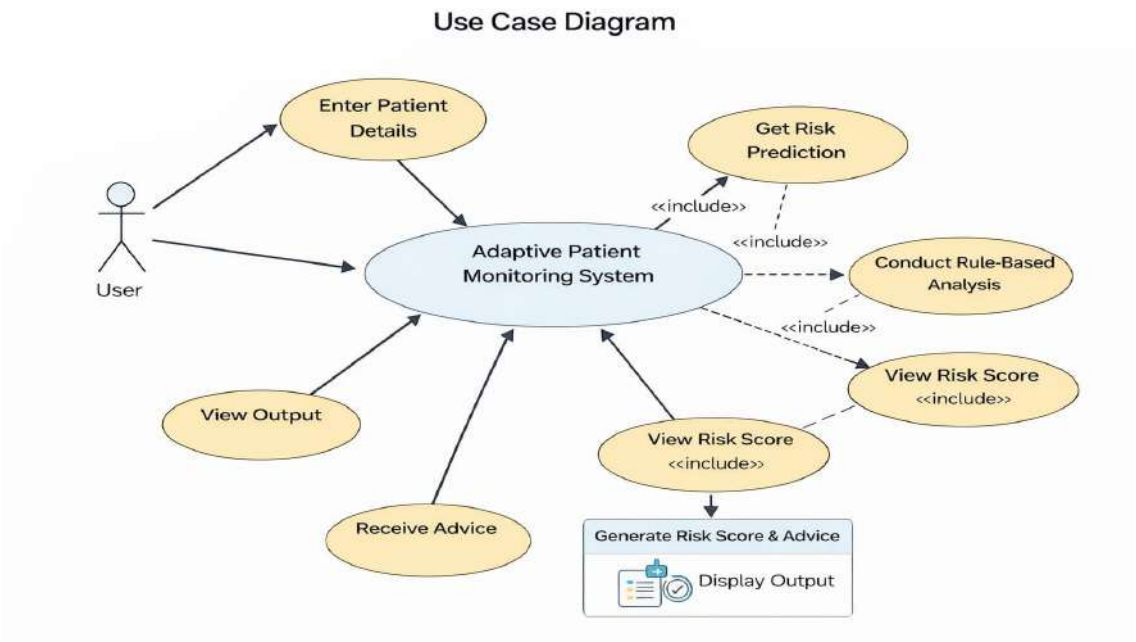


Design Diagrams :-

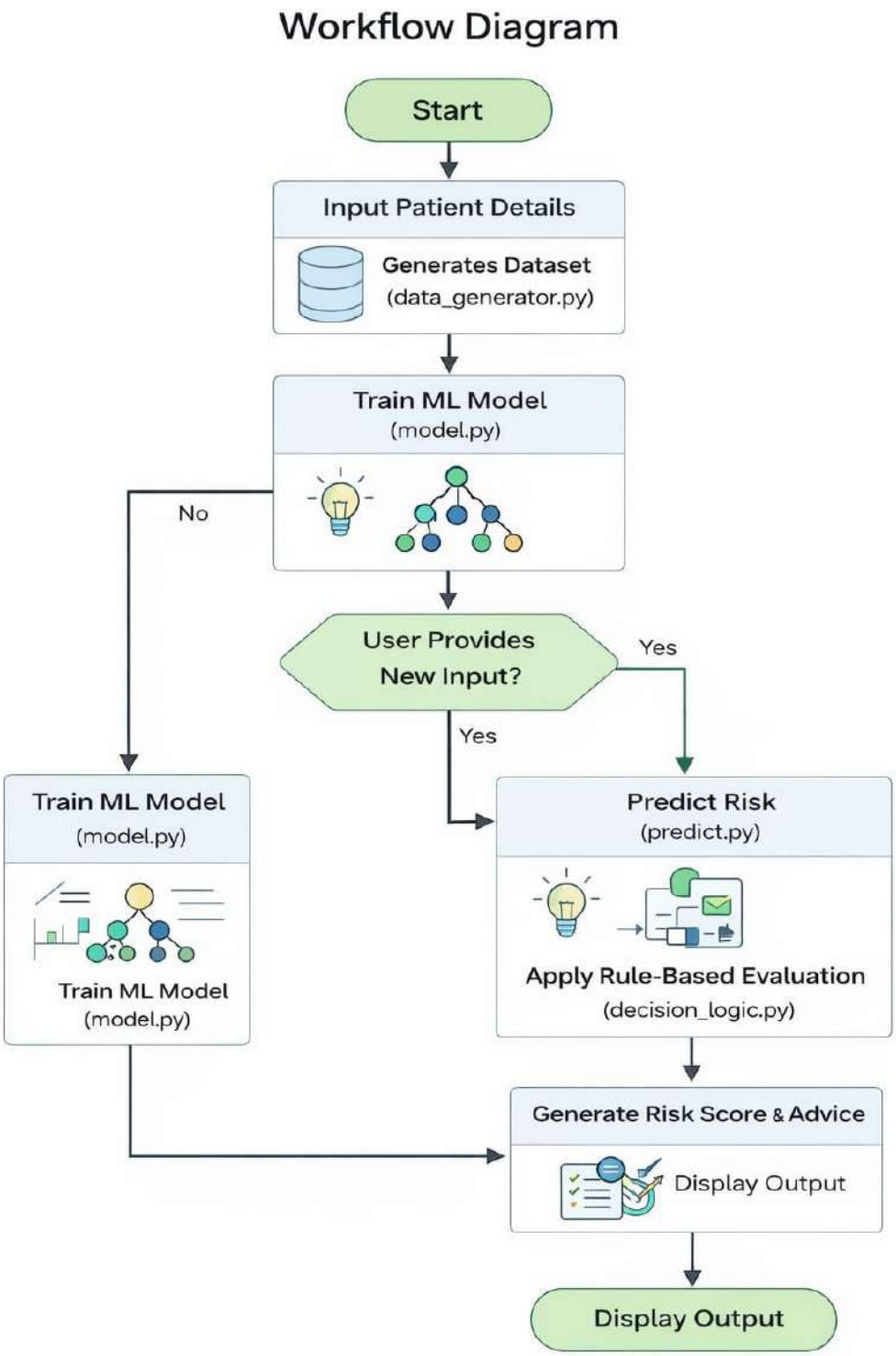
Include the following diagrams :-

1. Use Case Diagram
2. Workflow Diagram
3. Sequence Diagram
4. Class/Component Diagram

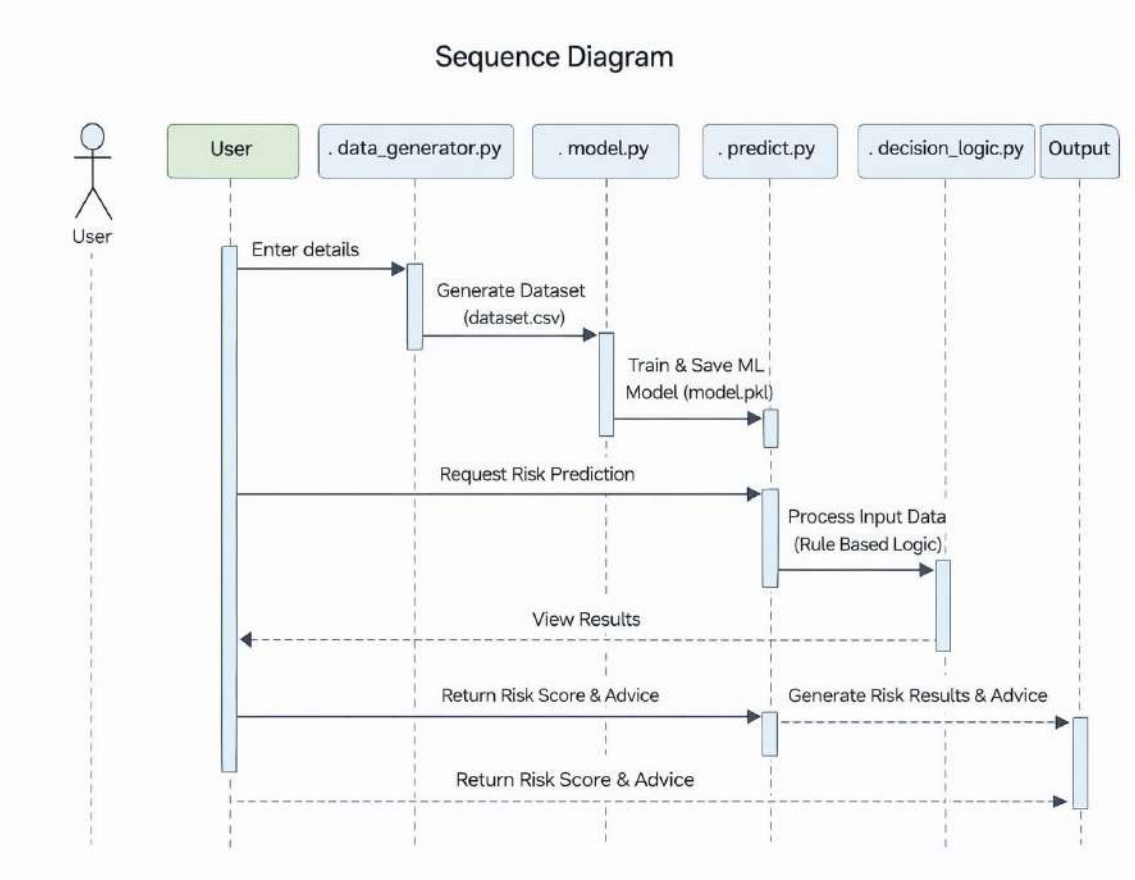
1. Use Case Diagram :-



2. Workflow Diagram :-

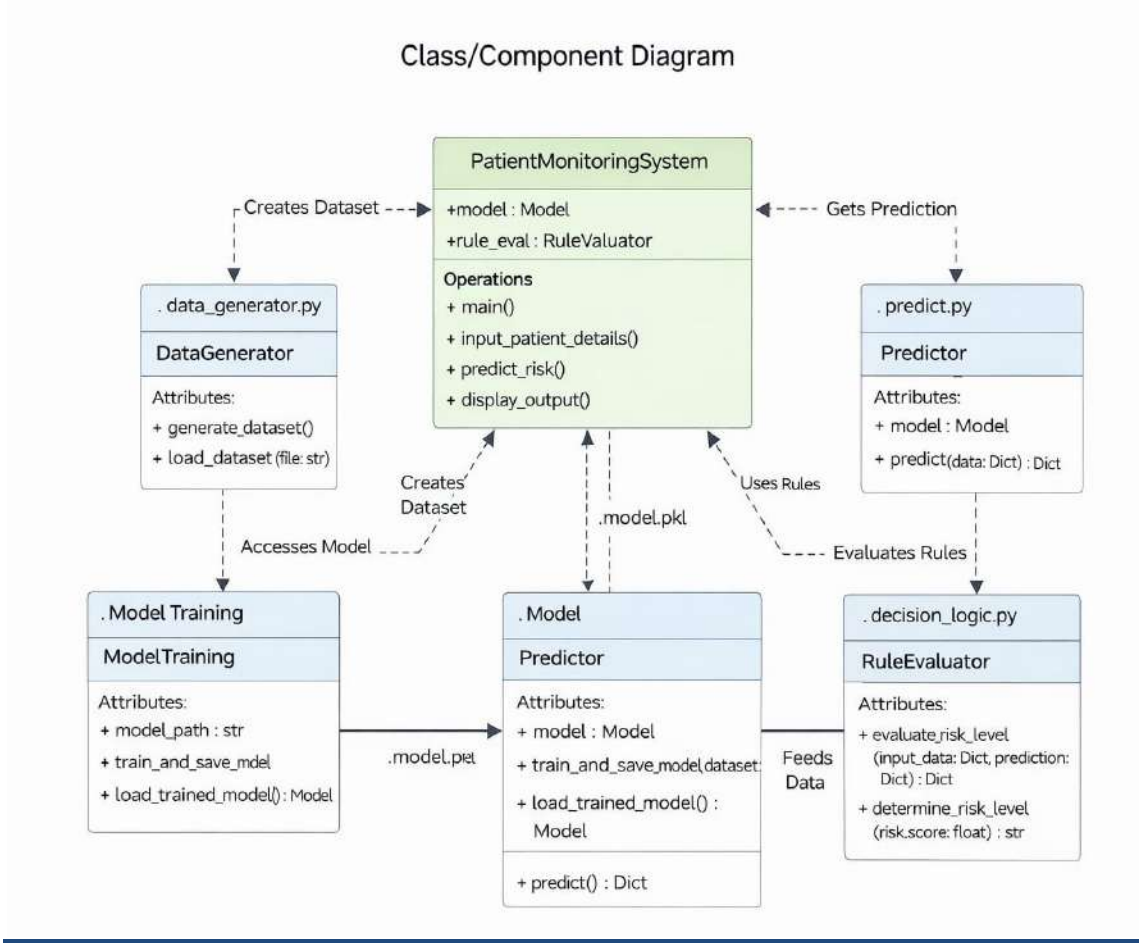


3. Sequence Diagram :-



- ***The sequence diagram shows how different modules interact step-by-step when a user enters patient data. The input is first processed by the data generator and model training modules, after which the prediction module generates a risk prediction. This prediction is then refined using rule-based logic. Finally, the system returns the risk level and advice to the user as output.***

4. Class/Component Diagram :-



The class/component diagram represents the structure of the system and how different modules are organized and connected. It shows components like data generator, model training, prediction, and decision logic working together under the main system. Each component has specific responsibilities such as data handling, model processing, and rule evaluation. Overall, it highlights the modular design and interaction between different parts of the system.

Methodology :-

1. ***Dataset generation using Python***
2. ***Data preprocessing***
3. ***Model training using Decision Tree***
4. ***Prediction using trained model***
5. ***Rule-based analysis***
6. ***Final output generation***

AI & ML Concepts Used :-

- ***Supervised Learning (Classification)***
- ***Decision Tree Algorithm***
- ***Feature-based prediction***
- ***Rule-based reasoning***
- ***Risk scoring mechanism***

- ✔ ***Machine Learning***

  Classification problem

  Decision Tree model

  Learns from dataset

- ✔ ***Artificial Intelligence***

  Rule-based decision making

  Condition-based reasoning

- ✔ ***Hybrid Approach***

  ML + Rules = More accurate system

Implementation Details :-

- ➢ ***Language: Python***
- ➢ ***Libraries: pandas, scikit-learn, pickle***
- ➢ ***Interface: Command Line Interface***

Files :-

- ***data_generator.py***
- ***model.py***
- ***predict.py***
- ***decision_logic.py***
- ***main.py***

Testing Approach/Strategy :-

- ***Tested with multiple input values***
- ***Checked edge cases (low/high values)***
- ***Verified consistency of predictions***
- ***Validated rule-based overrides***
- ***Compared ML vs final output***

Design Decisions & Rationale :-

- ***Used Decision Tree → easy to explain***
- ***CLI interface → simple and lightweight***
- ***Rule-based logic → safety layer***
- ***Decision Tree chosen for simplicity and interpretability***
- ***Rule-based logic added for safety-critical decisions***
- ***CLI used to keep system lightweight and simple***
- ***Modular design for better readability***

Sample Input :-

Age: 50
BP: 150
Heart Rate: 95
SpO2: 92
Temperature: 101

Sample Output :-

ML Prediction: Medium
Final Risk: High
Risk Score: 65
Advice: Emergency attention required

Sample Input And Output (Multiple Cases) :-

Case 1 (Low Risk) :-

Input :-

Age: 25
BP: 120
Heart Rate: 75
SpO2: 98
Temperature: 98

Output :-

ML Prediction: Low
Final Risk: Low
Risk Score: 15
Advice: Maintain healthy routine

Case 2 (Medium Risk) :-

Input :-

Age: 45
BP: 140
Heart Rate: 95
SpO2: 94
Temperature: 100

Output :-

ML Prediction: Medium
Final Risk: Medium
Risk Score: 45
Advice: Consult doctor

Case 3 (High Risk – Emergency) :-

Input :-

Age: 60
BP: 180
Heart Rate: 130
SpO2: 85
Temperature: 102

Output :-

ML Prediction: Medium
Final Risk: High
Risk Score: 75
Advice: Immediate medical attention required

Sample Results In Terminal CLI(Command Line) :-

```
--- Enter Patient Details ---

Age: 24
Gender (M/F): F
Blood Pressure: 110
Heart Rate: 72
SpO2: 98
BMI: 21
Body Temperature: 98
Smoker? (1 = Yes, 0 = No): 0
Activity (low/medium/high): high
Duration (days): 45
Symptom severity (1-10): 4

--- RESULT ---

ML Prediction: Low
Final Risk: Low
Risk Score: 0
Advice: Low risk: Maintain healthy routine

--- Enter Patient Details ---
Age: 62
Gender (M/F): M
Blood Pressure: 180
Heart Rate: 125
SpO2: 86
BMI: 31
Body Temperature: 102
Smoker? (1 = Yes, 0 = No): 1
Activity (low/medium/high): low
Duration (days): 5
Symptom severity (1-10): 8
```

```
--- RESULT ---

ML Prediction: High
Final Risk: High
Risk Score: 80
Advice: Emergency: Immediate medical attention
required
```

Engineering Practices Followed :-

✔ Modular coding

✔ Clear file structure

✔ Code separation (ML vs Logic)

✔ GitHub version control

Challenges Faced :-

- ***Understanding ML workflow***
- ***Data generation design***
- ***Balancing simplicity and functionality***
- ***Debugging input issues***
- ***Integrating ML with rule-based logic***

Real-World Impact :-

- ***Can assist in early diagnosis***
- ***Useful in telemedicine***
- ***Educational tool***

Learnings & Key Takeaways :-

- ***Practical understanding of ML models***
- ***Importance of data preprocessing***
- ***Combining AI and ML improves system reliability***
- ***Modular coding improves clarity***

Learning Outcomes :-

- ✓ ***Practical ML implementation***
- ✓ ***Importance of preprocessing***
- ✓ ***Real-world application of AI***
- ✓ ***Structured coding approach***

Future Enhancements :-

- ***Real hospital data integration***
- ***Web/mobile interface***
- ***Advanced ML models***
- ***Real-time monitoring***

Real World Application :-

- ***Health monitoring systems***
- ***Early risk detection tools***
- ***Decision support systems in healthcare***

Limitations :-

- ***Synthetic dataset used***
- ***Not medically certified***
- ***Limited feature set***

Future Scope :-

- ***Real-time sensor data***
- ***Deep learning models***
- ***Web app***
- ***Mobile integration***

Conclusion :-

This project demonstrates how machine learning and rule-based logic can be combined to create a meaningful healthcare decision-support system. It reflects the practical application of AI and ML concepts in solving real-world problems.

- ***Ethical Consideration :-***

  This system is for educational purposes and should not replace professional medical advice.

- ***Scalability :-***

  The system can be extended with more features and datasets.

References :-

- ***Scikit-learn documentation***
- ***Python official documentation***
- ***Basic ML course materials***

----------X----------